

TraFix

Reinforcement-learning traffic-signal control for a 5-junction
SUMO corridor

Code Review & Oral Defense

TraFix v6 · 6 presenters · June 2026





Agenda — six layers

1

AI model & training

v6 architecture · reward · 3-stage PPO

2

Inference & RuleGovernor

/telemetry_batch · 20-dim obs · safe-RL shielding

3

Live runner & safety net

actuation loop · starvation overrides · SUMO net

4

Test framework

121 mandatory tests · Type-1/2 · runner integrity

5

Metrics & results

12 metrics · honest accounting · figures

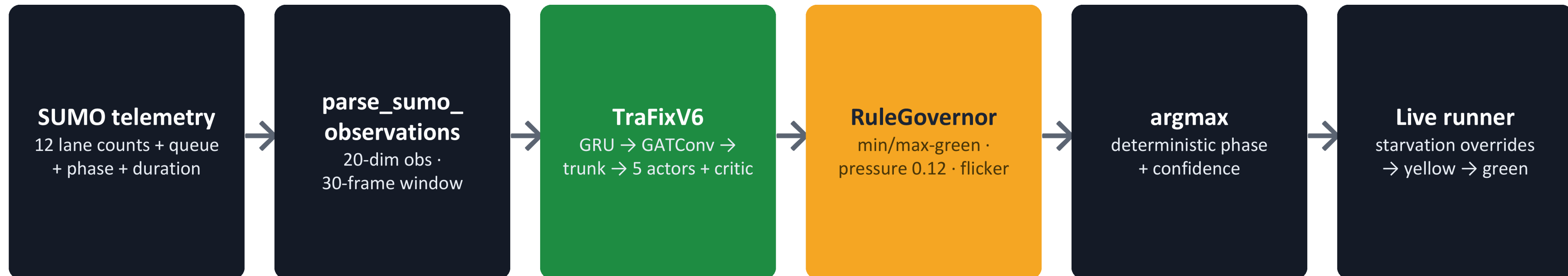
6

Frontend, ambulance, DB

dashboards · preemption · persistence · launchers



One decision, end to end



Every 10 simulated seconds, for each of 5 junctions: one of 6 phases. Trained in 3 stages (GRU pretrain → GAT pretrain → full PPO), ~101,476 parameters.



01

SECTION 1 · EREN KAAAN ORAL

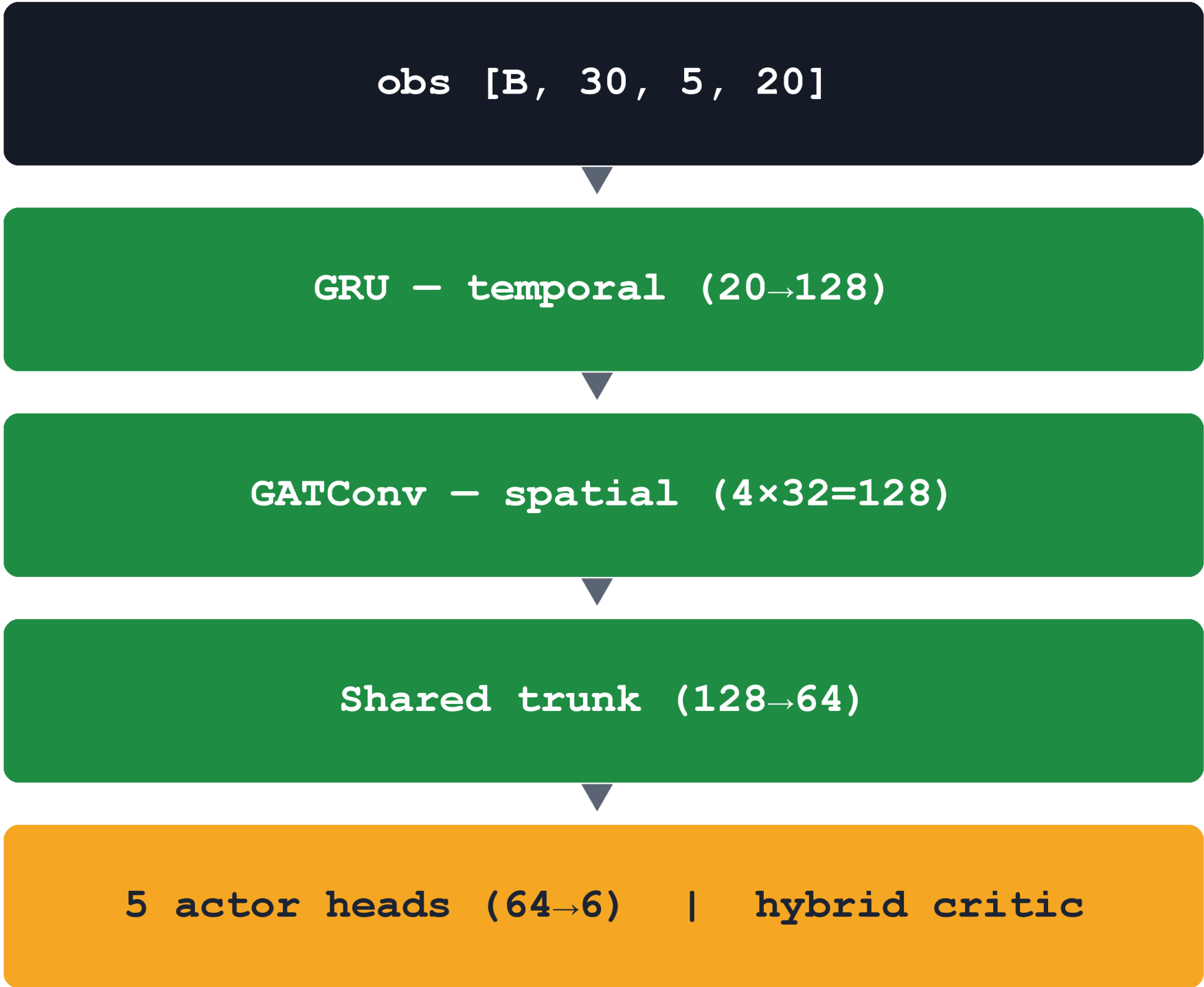
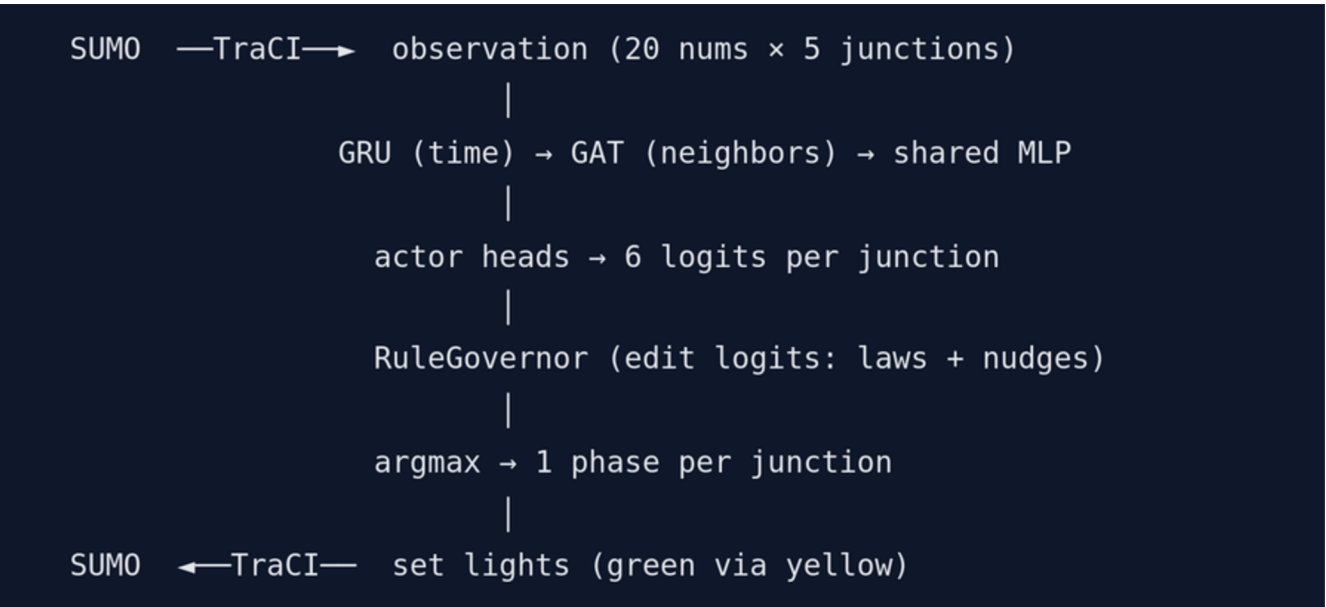
TraFix AI model & training

`trafix_v6/*.py · train_v6.py · backend/ai/trafix_v2.py (reward/GAE)`



The v6 architecture

- **Per-junction actor-critic.** ~101,476 parameters total.
- **GRU** encodes the 30-step time window per junction.
- **GATConv** coordinates the 5 junctions over a fixed bidirectional chain graph 0–1–2–3–4.
- **Shared trunk** → 5 actor heads + hybrid critic.
- **Hybrid critic:** $V_j = V_{\text{local}_j} + V_{\text{global}}$ (CTDE — centralised training, decentralised execution).





Forward pass — encode once



trafix_v6/trafix_v6.py

```
def _encode(self, obs, edge_index):
    B = obs.shape[0]
    h = self.temporal_enc(obs)                # GRU: [B,T,J,20] -> [B,J,128]
    h_flat = h.reshape(B * NUM_JUNCTIONS, self.hidden_dim)
    g = self.graph_enc(h_flat, self._batch_edge_index(edge_index, B)) # GATConv
    g = g.reshape(B, NUM_JUNCTIONS, -1)
    t = self.trunk(g)                         # [B,J,64]
    local_v = torch.stack([self.local_critics[j](t[:, j, :])
                           for j in range(NUM_JUNCTIONS)], dim=1)
    global_v = self.global_critic(t.mean(dim=1)).unsqueeze(1)
    v = (local_v + global_v).squeeze(-1)      # hybrid critic -> [B,J]
    return t, v
```

One shared GRU for all 5 junctions (batched B*J call) · **raw logits leave the model — the governor needs them un-softmaxed.**



Model lineage — why only v6 runs



GCN + multi-head attention (CoordinatedPPOAgent)

REMOVED — no trained weights; `trafix_v2.py` is now obs/reward/GAE infra only



GRU + GAT, 4 phases, global-only critic

DELETED from the repo (commit 76133fb)



GRU + GAT, 6 phases, hybrid local+global critic

THE ONLY RUNNING MODEL — `trafix_v6_final.pt` (ep 1999, best_reward -0.0292)



The reward function

- **Dense per-junction reward:** pressure, queue, throughput, green-wave, phase/wait penalties.
- **NEW anti-starvation term (−0.20):** per-movement, share-based — penalises demand sitting unserved, scaled by how long the phase held. Active even at low demand; zero when nothing is starved.
- **clear_bonus (+0.06):** rewards actually clearing cars — non-zero gradient at low flow.
- **fairness = 0.00:** computed but deliberately disabled (covered by starvation + governor + overrides).

term	weight
pressure	−0.30
queue	−0.25
throughput	+0.25
green_wave	+0.20
starvation	−0.20
phase_penalty	−0.08
clear_bonus	+0.06
wait_penalty	−0.05
fairness	0.00



Anti-starvation in code



backend/ai/trafix_v2.py

```
@dataclass
class RewardWeights:
    pressure=-0.30; queue=-0.25; throughput=0.25; fairness=0.00
    phase_penalty=-0.08; wait_penalty=-0.05; green_wave=0.20
    starvation=-0.20    # per-movement anti-starvation (active at low demand)
    clear_bonus=0.06    # low-demand shaping: reward clearing any car

# inside compute_reward, per junction:
group_demand = [N_thru+S_thru, N_left, S_left, E_thru+W_thru, E_left, W_left]
total_dem = sum(group_demand)
starvation = 0.0
if total_dem > 0:
    excess = min(dur / 45.0, 2.0)
    unserved_share = (total_dem - group_demand[phase]) / total_dem
    starvation = unserved_share * excess
```

Share-based → works with 1 car or 100. Zero when the held phase serves all demand — never punishes a correct hold.



3-stage training pipeline

Stage 1

GRU pretrain

Self-supervised next-step traffic prediction (no reward, random actions).
300 eps · LR 1e-3 → stage1_gru.pt



Stage 2

GAT + trunk pretrain

Neighbour-queue prediction; GRU frozen. 200 offpeak-only first.
400 eps · LR 3e-4 → stage2_*.pt



Stage 3

Full PPO

Pretrained encoders frozen 100 eps then unfrozen; differential LRs (GRU 1e-4 < GAT 2e-4 < heads 3e-4); cosine decay; governor active.
2000 eps → trafilix_v6_final.pt

Orchestrated by train_v6.py — pretraining gives stable encoders before the noisy RL signal arrives.



PPO + GAE — the update that trains it



trafix_v6/stage3_train_ppo.py

```
log_ratio = (new_lp - mb_old).clamp(-max_log_ratio, max_log_ratio)
ratio = torch.exp(log_ratio)
surr1 = ratio * mb_adv
surr2 = torch.clamp(ratio, 1.0 - clip_eps, 1.0 + clip_eps) * mb_adv
policy_loss = -torch.min(surr1, surr2).mean()
# clipped value loss + entropy bonus, with KL early-stop (target_kl=0.015)
total_loss = policy_loss + value_loss_coef * value_loss - entropy_coef * entropy_loss
```

0.99 / 0.95
 γ / λ

0.2
clip_eps

0.015
target_kl

0.5
grad clip

64
rollout

0.01→0.0005
entropy



02

SECTION 2 · PRESENTER 2

Inference path & RuleGovernor

`backend/main.py · trafix_v6/rule_governor.py`



Telemetry → decision

POST /telemetry_batch *build obs → parse*

30-frame rolling window *pre-filled on first call*

TraFixV6 forward *logits per junction*

RuleGovernor.apply *constrain logits*

softmax → argmax *phase + confidence*

- **Restart detection:** if batch.step goes backwards, clear the window + reset the governor (new SUMO run).
- **Heuristic fallback:** no weights loaded → rule-based hold/NS-vs-EW choice. Lights never freeze.
- **Persistence:** decisions + telemetry logged to Postgres on a background thread.



The v6 inference branch



backend/main.py

```
window_tensor = torch.stack(list(_v6_window)).unsqueeze(0)    # [1,T,5,20]
logits_list, _ = ai_agent(window_tensor)
obs_last = window_tensor[0, -1]
if _v6_governor is not None:
    logits_list = _v6_governor.apply(logits_list, obs_last)    # constrain logits
action_probs = torch.stack([torch.softmax(l, -1).squeeze(0)
                             for l in logits_list], dim=0)
# next_phase = argmax(action_probs[j]); confidence = max softmax prob
```

Why argmax at deploy? Deterministic, reproducible decisions for a live system — the same state always yields the same phase.



The 20-dim observation contract

idx	feature	normaliser
0–11	12 per-lane counts	÷ total-12-lane-sum (junction-relative share)
12	queue_length	÷ 200 (saturation cap)
13–18	current phase	6-bit one-hot
19	phase_duration	÷ 120, capped at 3.0

- **Scale-invariant:** 1 car out of 5 and 4 cars out of 20 both read as 0.20 — meaningful signal even at very low demand.
- **Shared everywhere:** parse_sumo_observations is used by every model version and every trainer.



parse_sumo_observations



backend/ai/trafix_v2.py

```
total_lane = sum(o.get(key, 0) for key in _LANE_KEYS)
denom = max(total_lane, 1) # floor at 1: no div-by-zero
for key in _LANE_KEYS:
    row.append(o.get(key, 0) / denom) # 0-11: junction-relative shares
row.append(o.get("queue_length", 0.0) / 200.0) # 12
one_hot = [0.0]*6; one_hot[int(o.get("current_phase", 0)) % 6] = 1.0
row.extend(one_hot) # 13-18
row.append(min(o.get("phase_duration", 0.0)/120.0, 3.0)) # 19
```

Fixed divisors ($\div 15/\div 30$) were removed — they starved the model of gradient signal at low demand, the root cause of argmax collapse.



The RuleGovernor — safe-RL shielding

Min-green *hard · stateless*

held < min_green → every other phase −1e9 (forced hold)

Max-green *hard · stateless*

held > max_green → current phase −1e9 (forced switch)

Pressure boost *soft · stateless*

bonus to busiest movement if > 12% of demand (was 35%)

Anti-flicker *soft · stateful*

A→B→A reversal gets −3.0 on the return logit

Phase-type aware (through 10/90 s, left 8/45 s) · applied in BOTH training and inference — the policy acts under the constraints it ships with. Masking, not reward shaping: illegal switches are impossible by construction.



The hard mask



trafix_v6/rule_governor.py

```
if duration < min_green:          # min-green: forbid every other phase
    for p in range(self.num_phases):
        if p != phase: mask[p] = _NEG_INF
elif duration > max_green:        # max-green: forbid staying
    mask[phase] = _NEG_INF
```

- **Three apply variants:** `apply()` (live/rollout, stateful flicker) · `apply_stateless()` (test runner) · `apply_stateless_batch()` (inside the PPO update).
- **Decodes straight from the obs:** `phase = argmax(obs[13:19])`, `duration = obs[19] × 120`.



They might ask...

Q1

What stops the AI doing something illegal/unsafe?

The governor masks it by construction every step (training + live) — shielding, not just reward shaping.

Q2

What happens with no model loaded?

Heuristic fallback: hold under min-green/no-demand, else pick NS vs EW by through-demand. Lights never freeze.

Q3

Why argmax, not sampling, at deploy?

Deterministic, reproducible decisions for a live system.



03

SECTION 3 · PRESENTER 3

Live runner, actuation & the safety net

`sumo/run_sumo_live.py · sumo/*.net.xml · sumo/*.sumocfg`



The live loop

- 1** **Advance yellows** — 3-step yellow, never interrupted
- 2** **Emergency preemption** — per-junction state machine runs first
- 3** **Every 10 steps: POST telemetry** — build batch → /telemetry_batch
- 4** **Apply returned phases** — via yellow transitions + min-green 10/8
- 5** **Fixed-time fallback** — backend unreachable 3× → 40-step NS/EW cycle



Starvation overrides — the safety net

8

Through-starvation

8 consecutive non-through decisions → force higher-demand through phase

10

Direction starvation

one direction monopolises 10 decisions while the other has cars → force it

15

Left-turn starvation

a left phase with demand unserved 15 decisions → force it

Mirrored in the test runner so tests measure the controller that ships. Goal of the argmax fix: make these rarely-triggered instead of load-bearing. (8/10/15 decisions ≈ 80/100/150 simulated seconds.)

⚠ TO BE UPDATED AFTER TRAINING

Override-trigger count from a live/test run showing they now fire rarely.



Left-turn starvation override

`tests/utils/sumo_runner.py`

```
for lp in (1, 2, 4, 5):
    dsl[tls_id][lp] = 0 if model_phase == lp else dsl[tls_id][lp] + 1
if model_phase not in (1, 2, 4, 5):
    # force the most-overdue left that still has demand
    if best_lp is not None and dsl[tls_id][best_lp] >= _LEFT_STARVE_LIMIT:
        model_phase = best_lp
```

- **Per-left-phase counters** reset whenever that left is served; only fire when demand exists.
- **Same constants live + test** (kept identical on purpose — an implicit contract across processes).

The SUMO network



Bidirectional chain 0–1–2–3–4 — the same graph the GATConv attends over

- **5 junctions, 3 lanes per approach;** 5 tlLogic × 12 phases (6 green + 6 yellow) = 60 phase entries.
- **Teleporting disabled** (--time-to-teleport -1) so gridlock stays visible in the reward and metrics — the model can't 'cheat' by having SUMO remove stuck cars.



They might ask...

Q1

"Your AI starved a left turn."

The runner has explicit STARVE / DIRECTION / LEFT guards; the retrained policy reduces how often they fire.

Q2

Why disable teleporting?

So the model can't cheat by having SUMO silently remove stuck cars — a gridlock stays a gridlock in the metrics.

Q3

How do lights never freeze?

Fixed-time fallback after 3 unreachable backend calls (40-step NS/EW cycle).



04

SECTION 4 · Alp Çağan İLHAN

Test framework

`tests/mandatory/* · tests/runners/* · tests/utis/sumo_runner.py`



The 121 mandatory tests

121

tests · run without SUMO
must stay green

- **FR tests:** telemetry ingestion, AI phase selection, emergency preemption, DB logging, yellow phase.
- **NFR tests:** inference latency, fallback, cross-platform.
- **Unit tests:** model (encoders, forward, checkpoint), observation, RuleGovernor, preemption.
- **Metrics-module tests:** every `compute_*` returns a well-formed dict.



External scenario tests — baseline vs AI

Type 1

Traffic intensity

low · medium · high

Type 2

Demand patterns

morning · evening · incident · pulse

Unseen

Stress scenarios

oscillating · supersaturation · stadium-exit · tidal-ramp ...

Every scenario runs twice — fixed-time baseline vs AI — through the same runner, the same metrics, the same integrity checks.



Runner integrity checks

```
tests/utils/sumo_runner.py

if mean_spd < _GRIDLOCK_SPEED_THRESHOLD:
    _low_speed_streak += 1
    if _low_speed_streak >= _GRIDLOCK_DURATION_S: _gridlock_detected = True
teleports = int(ET.parse(stats).getroot().find("teleports").get("total", 0)) # must be 0
```

- **Gridlock detection:** mean network speed < 0.5 m/s sustained for 300 s.
- **Teleport sanity:** authoritative total from statistics.xml — must be 0.
- **No survivorship bias:** vehicles still in the network at sim end are captured and folded into metrics.

⚠ TO BE UPDATED AFTER TRAINING

Type-1/Type-2 pass summary with the new model — 121 green, 0 teleports, 0 gridlock.



They might ask...

Q1

What makes a test "mandatory"?

They encode the FRs/NFRs and unit contracts; they are the CI gate and must stay green.

Q2

How is gridlock detected?

Sustained near-zero network speed (< 0.5 m/s for 300 s).

Q3

Why read teleports from statistics.xml?

It's the authoritative total; summary.xml is cumulative per-step and overcounts.



05

SECTION 5 · Kaan Turan

Metrics & results / figures

tests/metrics/* · tests/analysis/*



The 12 metrics

 Throughput & completion %

 Queue (halting + crawl-aware)

 Travel time

 Teleports

 Junction fairness

 Waiting time

 Time-loss

 Network speed

 Stops / vehicle

 Emissions

 Fuel consumption

 Pollutants

One `compute_<name>(sumo_output_dir)` per file, each returning a dict — composable and individually testable.



Honest accounting — why these definitions

`tests/metrics/throughput.py`

```
not_inserted = max(0, loaded - inserted)          # never even entered the net
cars_not_completed = still_running + not_inserted  # demand the controller failed to serve
completion_rate = arrived / total_demand
```

- **No survivorship bias:** travel time & throughput fold in cars still in the network at sim end — a gridlocking controller can't hide its victims.
- **Crawl-aware queue (< 5 km/h),** not just halting (< 0.1 m/s) → fair on continuous crawl, which halting-only undercounts.

⚠ TO BE UPDATED AFTER TRAINING

AFTER results tables/figures — throughput, completion %, slow-queue, travel time at low/medium/high vs the baseline and the old model.



They might ask...

Q1

How do you avoid survivorship bias?

Unfinished cars are recorded via TraCI and folded into the averages — demand the controller failed to serve counts against it.

Q2

Why a crawl-aware queue?

Halting-only (< 0.1 m/s) undercounts slow-moving congestion; < 5 km/h captures the crawl.

Q3

How is completion % computed?

arrived / total demand — including vehicles that were never even inserted.



06

SECTION 6 · PRESENTER 6

Frontend, ambulance, persistence & launchers

`frontend/* · sumo/emergency_preemption.py · backend/database.py · baslat.py`



Three frontends, one API

dashboard.html

Live junction states, chosen phases, model confidences

GET /state

emergency.html

Ambulance view — preemption events & transit metrics

GET /emergency_metrics

index.html

Architecture / entry page

GET /architecture

All three poll the FastAPI backend — no websockets, no state in the browser.



Ambulance / emergency preemption



sumo/emergency_preemption.py

```
class EmergencyPreemptionController:
    # stages: None -> "yellow" -> "allred" -> "green" -> "return_yellow" -> None
    def __init__(self, traci_mod, tls_ids, log, *, yellow_steps=3, all_red_steps=2,
                 max_emergency_green=60, stuck_speed=0.1, min_green_through=10):
        ...
    def is_active(self, tls_id): # AI/fallback skip this junction while active
        return self.stage.get(tls_id) is not None
```

Decoupled from the AI loop — the detection layer could later be swapped for CARLA+YOLO. Overrides the AI per junction during transit; metrics POSTed to /emergency_event.



Persistence & launch

Postgres schema (optional — backend runs fine without it)

`sessions`

one row per run, scenario + start/end

`step_log`

one row per junction per decision (logged off the request thread)

`emergency_log`

one row per completed preemption event

```
$ python baslat.py
# GUI on, model v6

$ python baslat.py --no-gui
# headless

starts uvicorn main:app
+ sumo/run_sumo_live.py
```

One-click demo: baslat.py launches the API and the live SUMO runner together.



They might ask...

Q1

How does an ambulance preempt the lights — and why decoupled?

Independent per-junction state machine via TraCI; swappable detection layer; overrides the AI for that junction while it transits.

Q2

What is persisted, and where?

Sessions, per-step decisions + telemetry, emergency events — in Postgres, if a DB exists (optional).

Q3

How do you launch the whole demo?

python baslat.py (GUI) or --no-gui.



Appendix — numbers to refresh after training

! TO BE UPDATED AFTER TRAINING

New checkpoint path & best_reward · diagnostic AFTER tables (type1_low, type1_medium) · high-traffic before/after (throughput, completion %, slow-queue, travel time) · override-trigger frequency · Type-1/Type-2 regression summary.

All other content in this deck is FINAL — structure, mechanism and methodology do not change with the retrain.

Coordination seams (so presenters don't contradict each other)

Governor — P2 owns mechanism; P1 says "we also train under the governor" · **Starvation overrides** — P3 owns them; P4 notes the test runner mirrors them · **Obs / phase mapping** — P2 owns the contract; P1 just cites the 20-dim input · **Argmax-fix narrative** — P1 owns it end-to-end; P3/P4/P5 know the one-liner



Questions?

TraFix v6 — the only running model. Trust the code (and `master_ai.md`).